

TWYGO • WORKSHOP • IA NA PRÁTICA

Apostila de conceitos - Workshop AI + API Twygo + CSV

Os conceitos do workshop em linguagem simples,
para acompanhar o desafio sem travar em nenhuma
palavra.



Esta apostila existe para explicar, em linguagem simples, o que cada coisa do projeto significa.

Ela não foi escrita para formar programadores em um dia. Ela foi escrita para a pessoa conseguir acompanhar o desafio sem travar em palavras como `API`, `.env`, `frontend`, `backend`, `token`, `CSV` e `localhost`.

Sumário

- **Parte 1 - Trabalhando com a IA** (seções 1 a 5): como usar Claude ou Codex, como pedir explicações melhores, a rodada "pergunte ao Claude", Skills e Rules, e o mapa mental do projeto.
- **Parte 2 - Conceitos do sistema** (seções 6 a 40): sistema local, frontend, backend, API, JSON, CSV, `.env`, token, portas, Git, erros comuns, testes, build e como os dados andam dentro do app.
- **Parte 3 - Na prática** (seções 41 a 45): o que cada arquivo faz, frases para pedir ajuda, regras de ouro, o kpi-boilerplate e o desafio do seu setor.
- **Parte 4 - Fechamento** (seções 46 a 48): checklist antes de pedir socorro, resumo ultra rápido e mensagem final.

1. Como vamos usar Claude ou Codex

Neste workshop, a pessoa não precisa decorar comandos nem saber programar sozinha.

A regra da aula é:

você dá prompts para o Claude/Codex → a IA altera o projeto → você confere o resultado

Quando aparecer um comando nesta apostila, ele serve para entender o que acontece por baixo. Na prática, o aluno pode pedir para a IA executar.

Exemplo:

Execute as ações necessárias para instalar as dependências e abrir o projeto local.
Depois me diga qual endereço eu devo abrir no navegador.

Outro exemplo:

Rode os testes do projeto e me explique o resultado em linguagem simples.

O aluno não precisa saber de memória o que é `npm install`, `npm run dev` ou `npx vite build`. Ele precisa saber pedir, conferir e explicar o que aconteceu.

2. Como pedir explicações melhores para a IA

Não use a IA apenas para fazer.

Use a IA para explicar.

Um bom pedido tem 5 partes:

1. O que você quer fazer.
2. O contexto do projeto.
3. O nível de explicação esperado.
4. O que a IA deve alterar.
5. Como a IA deve mostrar que funcionou.

Exemplo:

```
Quero entender como o projeto busca usuários na Twygo.  
Explique em linguagem simples.  
Mostre quais arquivos participam disso.  
Não altere nada ainda.  
No final, me diga como eu confiro se essa busca está funcionando.
```

Antes de deixar a IA alterar arquivos, você pode pedir:

```
Antes de alterar qualquer arquivo, explique seu plano.  
Diga o que você pretende fazer, por que isso é necessário e quais arquivos  
podem mudar.  
Use linguagem simples.
```

Depois que a IA alterar arquivos, peça:

```
Agora explique o que você mudou.  
Liste os arquivos alterados.  
Para cada arquivo, diga o que mudou, por que mudou e como eu confiro se  
funcionou.
```

Se a resposta vier técnica demais, peça:

```
Explica de novo como se eu nunca tivesse programado.  
Use uma comparação simples e um exemplo deste projeto.
```

Se você se perder no meio da aula, peça:

Resume tudo que já fizemos até agora.

Separe em:

1. o que já está funcionando
2. o que ainda falta
3. qual deve ser o próximo passo

3. Antes do projeto: a rodada "pergunte ao Claude"

Antes de abrir o projeto, cada pessoa faz uma rodada rápida com o Claude para descobrir como ele pode ajudar no próprio trabalho.

Como funciona

1. Abra o Claude (claude.ai ou o Claude Code, tanto faz nesta etapa).
2. Cole o pedido abaixo e complete com o seu setor e as suas tarefas.
3. Escolha os 2 usos que mais economizariam tempo para você.
4. Apresente para a turma em 1 minuto: a tarefa de hoje, como o Claude ajuda e qual é o ganho.

O pedido

Eu trabalho em [seu setor] e no dia a dia eu [suas 3 tarefas que mais tomam tempo].

Me dê 5 ideias de como você pode facilitar o meu trabalho.

Para cada uma: o que eu te pediria, um exemplo de pedido pronto e quanto tempo eu economizaria.

Use linguagem simples.

Exemplos de usos

- RH: resumir currículos, montar roteiro de onboarding.
- Financeiro: conferir notas e planilhas.
- Vendas: escrever o e-mail de proposta.
- Atendimento: rascunhar respostas de chamados.
- Gestão: transformar anotações de reunião em ata com próximos passos.

Regra da rodada

Não coloque dado real de colaborador ou de cliente no pedido. Use exemplos inventados.

Os usos que aparecerem na rodada viram ideias para o desafio do seu setor no final.

4. Skills e Rules: ensinando o seu agente

Você não precisa repetir as mesmas instruções em todo prompt. Dá para deixar o agente (Claude Code) "treinado" para o seu projeto com dois tipos de arquivo: Rules e Skills.

Rules (regras): o arquivo CLAUDE.md

Rules ficam em um arquivo de texto chamado `CLAUDE.md`, na pasta do projeto.

O agente lê esse arquivo toda vez que começa a trabalhar. É como um combinado fixo com ele.

Exemplos de regras:

- Responda sempre em português, sem jargão.
- Nunca mostre nem commite o token.
- Explique o plano antes de alterar arquivos.

Exemplo de um `CLAUDE.md` bem pequeno:

```
# Regras do projeto

- Responda sempre em português, sem jargão.
- Nunca mostre nem commite o token (TWYGO_API_TOKEN).
- Explique o plano antes de alterar arquivos.
- Depois de alterar, diga como eu confiro se funcionou.
```

Skills (receitas): a pasta .claude/skills

Skills ficam em arquivos `SKILL.md`, um por pasta:

```
.claude/skills/<nome>/SKILL.md
```

Uma skill é uma receita que o agente segue quando aparece uma tarefa específica.

Exemplos de skills:

- `rodar-sistema`: como instalar, ligar e conferir o sistema local.
- `nova-pagina`: como criar uma página nova no painel.
- `importar-planilha`: como ler um CSV novo e cruzar com os dados existentes.

Resumo em uma linha

Rule = o que ele SEMPRE respeita. Skill = COMO fazer uma tarefa específica.

Como pedir

Crie um arquivo CLAUDE.md neste projeto com regras para responder em português, sem jargão, nunca mostrar o token e explicar o plano antes de alterar arquivos.

5. Mapa mental do projeto

O sistema do workshop faz isto:

API da Twygo -> backend local -> frontend -> upload do CSV -> cruzamento dos dados -> gráficos

Em português bem direto:

1. A Twygo tem usuários cadastrados.
2. A API da Twygo entrega esses usuários.
3. Nosso backend local busca esses usuários com segurança.
4. Nosso frontend mostra os usuários na tela.
5. O aluno anexa um CSV de capacitação.
6. O sistema cruza API + CSV pelo email.
7. A tela mostra tabela, resumo e gráficos.

6. O que é um sistema local

Um sistema local é um sistema que roda no seu próprio computador.

Neste projeto, ele abre no navegador em:

`http://localhost:5183`

`localhost` significa:

`este computador aqui`

Ou seja, não é um site público na internet. É uma tela rodando na máquina da pessoa.

Exemplo no workshop

Quando você pede para o Claude/Codex abrir o projeto local, a IA liga duas partes no computador:

- a tela do sistema
- o servidor local

7. O que é frontend

Frontend é a parte visual do sistema.

É tudo aquilo que a pessoa vê e usa no navegador:

- botões
- textos
- tabelas
- campos de upload
- cards
- gráficos
- mensagens de erro

Exemplo no workshop

Quando o aluno abre:

```
http://localhost:5183
```

ele está vendo o frontend.

Neste projeto, o frontend foi feito com:

```
React + Vite
```

Tradução simples

Frontend é a vitrine do sistema.

Ele mostra as informações e recebe as ações da pessoa.

8. O que é React

React é uma ferramenta para criar telas.

Em vez de montar uma página inteira manualmente, usamos componentes.

Um componente é um pedaço reutilizável da tela.

Exemplos de componentes:

- um card de resumo
- uma tabela
- um gráfico

- uma mensagem de erro
- um botão

Exemplo no workshop

O painel de capacitação é uma tela React.

Ele mostra:

- usuários vindos da Twygo
- dados vindos do CSV
- cruzamento entre usuários e capacitações
- gráficos por setor, área, categoria e "Concluiu x não concluiu"

9. O que é Vite

Vite é uma ferramenta que ajuda a rodar o frontend.

Ele faz o React abrir rápido no navegador durante o desenvolvimento.

Exemplo no workshop

Quando a IA liga o frontend, o Vite abre a tela em:

```
http://localhost:5183
```

Tradução simples

Vite é o motor que liga a tela durante a aula.

10. O que é backend

Backend é a parte do sistema que fica por trás da tela.

Ele não é feito para o usuário final clicar. Ele é feito para processar dados, proteger informações e conversar com outros sistemas.

Neste projeto, o backend faz uma coisa muito importante:

```
buscar usuários na API da Twygo sem colocar o token dentro da tela
```

Exemplo no workshop

O backend roda em:

```
http://localhost:5184
```

Ele tem uma rota:

```
GET /api/users
```

Quando o frontend chama essa rota, o backend busca os usuários na Twygo.

Tradução simples

Backend é a cozinha do restaurante.

O usuário vê o prato na mesa, mas não vê a cozinha trabalhando.

11. O que é Node.js

Node.js permite rodar JavaScript fora do navegador.

Normalmente, JavaScript roda dentro da página. Com Node.js, conseguimos usar JavaScript também no servidor.

Exemplo no workshop

O backend local foi feito com:

```
Node.js + Express
```

Isso permite criar rotas como:

```
/health  
/api/users
```

12. O que é Express

Express é uma ferramenta para criar backend com Node.js.

Ele ajuda a criar rotas.

Uma rota é um endereço que responde alguma coisa.

Exemplo no workshop

Rota para testar se o servidor está vivo:

```
GET /health
```

Resposta esperada:

```
{"ok":true}
```

Rota para buscar usuários:

```
GET /api/users
```

Resposta esperada:

```
lista de usuários da Twygo
```

13. O que é API

API é uma forma de um sistema conversar com outro sistema.

Uma pessoa usa tela.

Um sistema usa API.

Exemplo simples

Quando você entra em um aplicativo de clima, ele provavelmente conversa com uma API para buscar a temperatura.

No nosso caso, o sistema conversa com a API da Twygo para buscar usuários.

Exemplo no workshop

O backend local chama:

```
https://api.twygo.com/api/v2/users
```

Essa API devolve usuários cadastrados na plataforma.

14. O que é endpoint

Endpoint é um endereço específico dentro de uma API.

Pense na API como um prédio.

O endpoint é uma sala dentro desse prédio.

Exemplo no workshop

API base:

```
https://api.twygo.com
```

Endpoint de usuários:

```
/api/v2/users
```

Endereço completo:

```
https://api.twygo.com/api/v2/users
```

Paginação: a API entrega os dados em páginas

A API não devolve todos os usuários de uma vez. Ela entrega em **páginas**:

```
https://api.twygo.com/api/v2/users?page=1&per_page=50
```

- `page=1` é a primeira página.
- `per_page=50` traz 50 usuários por página (o máximo é 100).
- A resposta traz `pagination.total_pages` (quantas páginas existem) e `pagination.total_entries` (quantos usuários existem no total).

No workshop são **186 usuários em 4 páginas** de 50. Se o sistema buscar só a página 1, vêm só 50 usuários e o cruzamento com o CSV fica incompleto. O projeto do workshop já busca **todas as páginas** (`/api/users?all=true`) antes de cruzar.

15. O que é método HTTP

Método HTTP é o tipo de ação que você quer fazer em uma API.

Os mais comuns são:

- `GET` : buscar dados
- `POST` : criar dados
- `PUT` : atualizar dados
- `DELETE` : apagar dados

No workshop

Usamos principalmente:

GET

Porque queremos buscar usuários.

Não queremos criar, editar ou apagar usuários da Twygo durante a aula.

16. O que é JSON

JSON é um formato de dados muito usado em APIs.

Ele parece um objeto com chaves e valores.

Exemplo

```
{  
  "name": "Ana Silva",  
  "email": "ana@empresa.com",  
  "area": "Marketing"  
}
```

Tradução simples

JSON é uma forma organizada de enviar informações entre sistemas.

17. O que é CSV

CSV é uma planilha em formato simples.

Cada linha representa um registro.

Cada coluna é separada por vírgula.

Exemplo

```
email,area,categoria,curso,horas_capitacao,status_capitacao,nota,concluido_em  
ana@empresa.com,Marketing,Produto,Curso de onboarding,6,Concluido,9.1,2026-05-  
10
```

No workshop

O CSV do workshop se chama `capitacao_workshop.csv`. Ele contém dados de capacitação: 287 linhas e 160 e-mails diferentes. Os e-mails são **reais** da Twygo; os cursos, horas e notas são de exemplo. Use o arquivo só no workshop e não compartilhe fora dele.

Ele é cruzado com os usuários da Twygo usando o email.

18. O que é cruzamento de dados

Cruzamento de dados é juntar duas listas usando uma informação em comum.

Neste projeto, a informação em comum é:

email

Exemplo

API da Twygo:

ana@empresa.com – Ana Silva

CSV:

ana@empresa.com – 6 horas de capacitação

Resultado cruzado:

Ana Silva – Marketing – 6 horas de capacitação

Como saber se deu certo (números esperados)

Com o CSV do workshop, a tela deve mostrar mais ou menos isto:

O quê	Número esperado
Usuários vindos da API	186
Linhas do CSV	287
E-mails diferentes no CSV	160
E-mails que cruzam (todas as páginas)	150
E-mails que cruzam se buscar só a página 1	50
E-mails que nunca cruzam (de propósito)	10, de <code>pessoa01</code> a <code>pessoa10@anonimizado.com</code>
Usuários da Twygo sem nenhuma capacitação no CSV	36

Pessoas e linhas são coisas diferentes: uma pessoa pode ter vários cursos, então aparece em várias linhas do CSV. São 150 **pessoas** cruzadas, mas 271 **linhas** cruzadas.

Os 10 e-mails de teste (`peessoa01` a `peessoa10@anonimizado.com`) estão no CSV de propósito, para você ver como o sistema mostra quem **não** foi encontrado na Twygo.

Outros números que podem parecer estranhos:

- **"Sem setor" e "Sem area no perfil"** aparecem bastante porque muitos usuários não têm o setor preenchido na Twygo. Isso é dado real, não erro do sistema.
- **Horas totais:** o painel do exercício mostra 2020h, que são só as horas das linhas cruzadas. A planilha inteira tem 2.134h; a diferença são as 16 linhas que não cruzam.

19. O que é .env

`.env` é um arquivo de configuração local.

Ele guarda informações que mudam de pessoa para pessoa ou que não devem ficar públicas.

Exemplos:

- token da API
- endereço base da API
- porta do servidor

Exemplo no projeto

Arquivo:

```
.env
```

Conteúdo:

```
TWYGO_API_TOKEN=cole_o_bearer_token_aqui  
TWYGO_API_BASE_URL=https://api.twygo.com  
PORT=5184
```

Por que isso é importante

O token é uma chave de acesso.

Ele não deve ficar escrito dentro do código da tela.

Ele também não deve ser publicado no GitHub.

20. O que é .env.example

`.env.example` é um modelo do arquivo `.env`.

Ele mostra quais variáveis precisam existir, mas sem colocar o token real.

Exemplo

```
TWYGO_API_TOKEN=cole_o_bearer_token_aqui  
TWYGO_API_BASE_URL=https://api.twygo.com  
PORT=5184
```

Como usar com Claude ou Codex

Regra do workshop: o token **nunca** vai no chat com a IA. É você quem cola o token no arquivo.

Peça:

```
Crie o .env a partir do .env.example e abra o arquivo para eu colar o token.  
Não peça nem mostre o token no chat.
```

A IA abre o arquivo para você (no Mac, com `open -e .env` ; no Windows, com `notepad .env`).
Aí é só:

1. Colar o token logo depois de `TWYGO_API_TOKEN=` , sem espaços.
2. Salvar o arquivo (Cmd+S ou Ctrl+S) e fechar.
3. Avisar a IA: "pronto, coleí o token".

Se você colou o token no chat sem querer, avise o instrutor: o token do workshop é desativado no final.

21. O que é token

Token é uma chave de acesso.

Ele serve para provar para a API que você tem permissão para buscar dados.

Exemplo simples

Pense no token como um crachá.

Sem crachá, você não entra.

Sem token, a API não entrega os dados.

22. O que é Bearer token

Bearer token é um tipo comum de token usado em APIs.

Quando uma API pede Bearer token, normalmente o pedido vai assim:

```
Authorization: Bearer seu_token_aqui
```

No workshop

O aluno não precisa escrever essa linha manualmente.

O backend monta isso usando o valor do `.env`.

23. O que é segurança do token

O token deve ficar protegido.

Não coloque token real em:

- print de tela
- WhatsApp aberto
- README público
- código do frontend
- GitHub
- slide compartilhado publicamente

Regra simples

Token real fica no `.env`.

Token de exemplo fica no `.env.example`.

24. O que é porta

Porta é como se fosse uma entrada específica do computador.

O mesmo computador pode rodar vários sistemas ao mesmo tempo, cada um em uma porta diferente.

No workshop

Frontend:

```
http://localhost:5183
```

Backend:

```
http://localhost:5184
```

25. O que é CORS

CORS é uma regra de segurança do navegador.

Ela controla se uma tela pode conversar com outro endereço.

Por que aparece no workshop

O frontend roda em:

```
localhost:5183
```

O backend roda em:

```
localhost:5184
```

Mesmo estando no mesmo computador, para o navegador são endereços diferentes.

Por isso o backend precisa permitir essa conversa.

26. O que é npm

`npm` é o gerenciador de pacotes do Node.js.

Ele instala ferramentas e bibliotecas usadas no projeto.

Como pedir para a IA usar npm

Para instalar dependências:

```
Instale as dependências do projeto.
```

Para abrir o projeto:

```
Rode o projeto local e me diga o link que devo abrir no navegador.
```

Para rodar testes:

```
Rode os testes e me explique se passou ou se falhou.
```

27. O que é package.json

`package.json` é a ficha técnica do projeto.

Ele diz:

- nome do projeto
- scripts disponíveis
- bibliotecas usadas
- versão das dependências

Exemplo no workshop

Scripts importantes:

```
{
  "dev": "concurrently \"npm:server\" \"npm:client\"",
  "client": "vite --host 127.0.0.1 --port 5183",
  "server": "node server/index.js",
  "test": "vitest run"
}
```

28. O que é node_modules

`node_modules` é a pasta onde ficam as dependências instaladas pelo `npm install`.

Ela costuma ser grande.

Ela não deve ser editada manualmente.

Ela também não deve ser enviada para o GitHub.

Regra simples

Se a pasta `node_modules` sumiu ou o projeto reclamar de dependência, peça para a IA:

```
Reinstale as dependências do projeto e explique o que foi feito.
```

29. O que é Git

Git é uma ferramenta para guardar o histórico do projeto.

Ele registra mudanças em commits.

Exemplo

Um commit é como uma foto do projeto naquele momento.

Você pode ver:

- o que mudou
- quando mudou
- quem mudou
- qual mensagem explica a mudança

30. O que é GitHub

GitHub é um lugar online para guardar repositórios Git.

Git é a ferramenta.

GitHub é o site onde o projeto fica hospedado.

No workshop

O repositório guarda:

- código do app
- desafio
- apostila
- CSV de exemplo
- instruções de uso

31. O que é branch

Branch é uma linha separada de trabalho dentro do Git.

No workshop

Temos uma branch para os alunos:

```
alunos
```

A ideia é deixar essa branch pronta para a turma usar sem bagunçar a branch principal.

32. O que é commit

Commit é um pacote de alterações salvo no histórico do Git.

Exemplo de mensagem

```
docs: add concepts booklet
```

Essa mensagem quer dizer:

```
adicionei uma documentação/apostila de conceitos
```

33. O que é erro 401

Erro `401` normalmente significa falta de autorização.

No workshop

Se aparecer `401`, pode ser:

- token errado
- token vencido
- token não foi colocado no `.env`
- backend não conseguiu ler o `.env`

34. O que é erro 404

Erro `404` significa que o endereço não foi encontrado.

Exemplo

Se você abrir:

```
http://localhost:5184/usuarios
```

mas o projeto só tem:

```
/api/users
```

pode dar `404`.

35. O que é erro 500

Erro `500` significa erro no servidor.

No workshop, pode acontecer se:

- a API externa falhar
- o backend quebrar
- alguma configuração estiver errada

36. O que é loading

Loading é o estado de carregamento.

Ele aparece quando o sistema ainda está buscando dados.

Exemplo

Quando a tela abre, ela pode mostrar algo como:



Carregando usuários...

Isso evita que a pessoa ache que o sistema travou.

37. O que é estado vazio

Estado vazio é quando não tem dados para mostrar.

Exemplo

Se o aluno ainda não anexou o CSV, a tela pode mostrar uma mensagem dizendo que precisa anexar a planilha.

Isso é melhor do que deixar a tela em branco.

38. O que é teste automatizado

Teste automatizado é um código que verifica se outra parte do sistema continua funcionando.

Exemplo no workshop

Os testes podem conferir:

- se o CSV é lido corretamente
- se os usuários são cruzados pelo email
- se os cálculos de horas estão certos
- se os dados dos gráficos estão corretos

Como pedir

Rode os testes automatizados do projeto e me diga quantos passaram.

39. O que é build

Build é gerar a versão final do frontend.

Durante a aula usamos o modo de desenvolvimento.

No build, o Vite prepara arquivos otimizados.

Como pedir

Gere o build do projeto e me diga se apareceu algum erro.

Se o build falha, pode existir algum erro que impediria o sistema de ir para produção.

40. Como os dados andam dentro do app

Fluxo completo:

1. Aluno abre `http://localhost:5183`
2. Frontend carrega a tela
3. Frontend chama `/api/users`
4. Backend recebe esse pedido
5. Backend lê o token do `.env`
6. Backend chama a API da Twygo
7. API da Twygo devolve usuários
8. Backend devolve usuários para o frontend
9. Aluno anexa o CSV
10. Frontend lê o CSV
11. Frontend cruza CSV + usuários pelo email
12. Frontend mostra tabela, cards e gráficos

41. O que cada arquivo principal faz

`README.md`

Explica como usar o projeto.

`APOSTILA_CONCEITOS.md`

Explica os conceitos para quem nunca programou.

`.env.example`

Modelo das configurações necessárias.

`package.json`

Lista as ações que a IA pode executar e as dependências do projeto.

`server/index.js`

Liga o backend local.

`server/twygoApi.js`

Concentra a chamada para a API da Twygo.

`src/App.jsx`

Tela principal do frontend.

`src/trainingCsv.js`

Lê e interpreta o CSV.

`src/trainingDashboard.js`

Cruza dados e calcula indicadores.

`public/capitacao_workshop.csv`

CSV de exemplo usado na aula.

42. Frases que ajudam a pedir ajuda para a IA

Quando der erro, não mande apenas:

não funcionou

Mande assim:

Estou usando o projeto do workshop com Claude/Codex.
Pedi para abrir o projeto local.
Esperava abrir a tela em `http://localhost:5183`.
Mas apareceu este erro:
[cole o erro completo aqui]
Me explique em linguagem simples e corrija.

Quando não entender um conceito:

Explique o que é backend usando o exemplo deste projeto.
Fale como se eu nunca tivesse programado.

Quando quiser confirmar se está certo:

Confira se meu .env está no formato correto, mas não mostre nem repita meu token real.

43. Regras de ouro: nada sai da sua máquina

As três regras de segurança

1. **Roda só no seu PC.** Tudo o que fazemos no workshop roda em `localhost`. Nenhum sistema sobe para a internet, para um servidor ou para a nuvem.
2. **Repositório privado.** Se for salvar o código, salve só no repositório privado do time. Nunca em um repositório público.
3. **Auditoria antes de publicar.** Antes de qualquer coisa ir para o ar, João, Adriana ou um dev revisam os números, os acessos e a segurança.

A IA escreve rápido, mas não responde por erro de número nem por dado exposto.

Cuidado com os dados

- Dados de colaboradores são dados pessoais (LGPD).
- Não envie planilhas reais para sites públicos ou IAs públicas.
- Não tire prints com token ou dados sensíveis.
- Use um token só para o workshop.

As regras da turma

1. **Errar é de graça.** Está tudo no seu computador. Se quebrar, a gente arruma.
2. **Entenda o fluxo.** Mais importante que o código é saber o caminho que os dados fazem.
3. **Pergunte antes de sofrer.** Travou por 5 minutos? Chama o instrutor.
4. **Fale em português.** Pode pedir para a IA em português, do seu jeito.

44. Depois do workshop: use o kpi-boilerplate como base

O `kpi-boilerplate` é um esqueleto pronto para você criar seus próprios painéis de indicadores sem começar do zero.

O que já vem pronto

- Frontend em React + TypeScript (Vite).

- Backend em Python com FastAPI.
- Banco de dados PostgreSQL.
- Layout com menu lateral (sidebar).
- Cards de KPI.
- Gráficos.
- Tabela com exportação para CSV.
- API + banco já conectados.
- `CLAUDE.md` com as regras do projeto e a skill `rodar-sistema`.
- Tudo rodando via Docker.

Tradução simples: o restaurante

- **PostgreSQL** é o estoque: onde os dados ficam guardados.
- **Python/FastAPI** é a cozinha: busca os dados no estoque e prepara.
- **React** é o salão: onde a pessoa vê e usa o painel.

Como começar

Você precisa ter instalado:

- Docker Desktop
- Git
- Claude Code

Depois, os passos são:

```
git clone https://github.com/Twygo/kpi-boilerplate
cd kpi-boilerplate
cp .env.example .env
make up
```

Com o sistema ligado, abra:

- `http://localhost:5173` - o painel
- `http://localhost:8000/docs` - a API

Para desligar:

```
make down
```

Lembre: você também pode pedir para o Claude Code fazer esses passos por você.

Exemplo de primeiro prompt

Crie uma página Turnover no menu, com cards e um gráfico mensal, lendo a planilha turnover.csv que vou te passar.

Acesso ao repositório

Para ter acesso ao repositório, fale com o João.

As regras de ouro continuam valendo

Tudo roda local, e nada sobe sem auditoria.

45. Desafio do seu setor (bônus)

Escolha uma tarefa repetitiva ou manual da sua área e faça uma demonstração simples com o Claude Code.

Comece pelo boilerplate-workshop

O `boilerplate-workshop` é um painel pronto feito para este desafio. Ele roda só com Node, sem Docker e sem banco de dados, e só no seu computador.

- **Minha planilha:** carregue um CSV ou Excel e veja cards, gráfico e tabela.
- **Usuários Twygo:** lista os usuários da API, com o token protegido no backend.
- **Cruzar planilha x Twygo:** junta qualquer planilha com os usuários pelo e-mail.
- **Minha automação:** página modelo para o seu desafio.
- O `CLAUDE.md` e as skills já explicam as regras para o agente, e o `PROMPTS.md` traz pedidos prontos por área.

Primeiro pedido no Claude Code, numa pasta nova:

Clone <https://github.com/ASP-J/workshop-boilerplate.git>, leia o `CLAUDE.md` e o `README`, instale o que precisar, rode o sistema e me diga qual endereço abrir.

O código está em <https://github.com/ASP-J/workshop-boilerplate> e também na pasta `06-boilerplate-workshop` do kit.

O sistema abre em <http://localhost:5193>. Ele precisa do Node 22.12 ou mais novo.

Se depois o projeto crescer e precisar de banco de dados, use o `kpi-boilerplate` (seção 44).

O que apresentar

1. Setor.

2. Problema escolhido.
3. Como é feito hoje.
4. Como você imaginou a automação.
5. O que você conseguiu criar com IA.
6. Demonstração rápida.
7. Ganho se virasse real.

Ideias por setor

- **RH:** onboarding de novos colaboradores; cruzar treinamentos.
- **Financeiro:** conferir planilhas; classificar despesas.
- **Vendas:** resumir leads; montar proposta.
- **Atendimento:** resumir chamados.
- **Marketing:** calendário de campanhas.
- **Produto:** organizar feedbacks.
- **Engenharia:** checklist de testes.
- **BI:** CSV virando dashboard.

46. Checklist para o aluno antes de pedir socorro

Antes de chamar o instrutor, conferir:

1. Pedi para a IA instalar as dependências?
2. Colei o token no `.env`, depois de `TWYGO_API_TOKEN=` (sem passar pelo chat)?
3. Salvei o arquivo `.env` e avisei a IA?
4. Pedi para a IA abrir o projeto local?
5. Abri `http://localhost:5183` ?
6. O backend abriu em `http://localhost:5184/health` ?
7. O CSV tem a coluna `email` ?
8. O email do CSV bate com o email da Twygo?
9. Copiei o erro inteiro?
10. Tentei explicar o que eu esperava que acontecesse?

47. Resumo ultra rápido

Frontend

A tela que o usuário vê.

Backend

A parte escondida que busca dados e protege o token.

API

Forma de um sistema conversar com outro.

`Endpoint`

Endereço específico dentro de uma API.

`Token`

Chave de acesso.

`.env`

Arquivo local onde ficam configurações sensíveis.

`CSV`

Planilha simples em texto.

`JSON`

Formato comum de resposta de API.

`localhost`

O próprio computador.

`porta`

Entrada específica para acessar um sistema local.

`Git`

Histórico do projeto.

`GitHub`

Lugar online onde o repositório fica salvo.

`Rule (CLAUDE.md)`

O que o agente sempre respeita.

`Skill`

Receita de como o agente faz uma tarefa específica.

48. Mensagem final para a turma

Você não precisa decorar tudo.

O objetivo é entender o caminho:

tela -> backend -> API -> dados -> CSV -> cruzamento -> gráficos

Se você entendeu esse caminho, já entendeu a parte mais importante do workshop.